L4 - Multiprocessing

The serial execution time for the chunked Mandelbrot implementation took 4.49 seconds when a resolution of 8192x8192 was used. The sweep was run with the same resolution as the serial implementation from 2 to 16 cores, of which the last 8 were logical.

As can be seen in Table 1, and on Figure 1 and Figure 2, runs with an odd number of cores seems to perform worse than runs with an even number of cores. This is because 8192 cannot be neatly divided into odd chunks of the same size, and due to how the mandelbrot_parallel function partitions it results in a very small straggler task that one of the available cores must compute.

For three cores the partitioning mechanism results in the following number of rows: 2730, 2730, 2730, 2, where the last partition has a large amount of overhead compared to the amount of compute. This could be fixed by simply adding the fourth partition to the third to avoid the overhead extra overhead.

Number of cores	Time [s]	Speedup	Efficiency
1	4.4918	0	1.0
2	2.6497	1.6952	0.8476
3	3.6115	1.2438	0.4146
4	2.3212	1.9351	0.4838
5	2.4860	1.8069	0.3614
6	1.8847	2.3834	0.3972
7	1.8886	2.3784	0.3398
8	1.5423	2.9125	0.3641
9	1.5078	2.9791	0.3310
10	1.3087	3.4323	0.3432
11	1.2587	3.5686	0.3244
12	1.1538	3.8930	0.3244
13	1.0874	4.1309	0.3178
14	1.0025	4.4808	0.3201
15	0.9591	4.6834	0.3122
16	0.9099	4.9368	0.3086

Table 1, The execution time, speedup and efficiency measured for 1 to 16 cores.

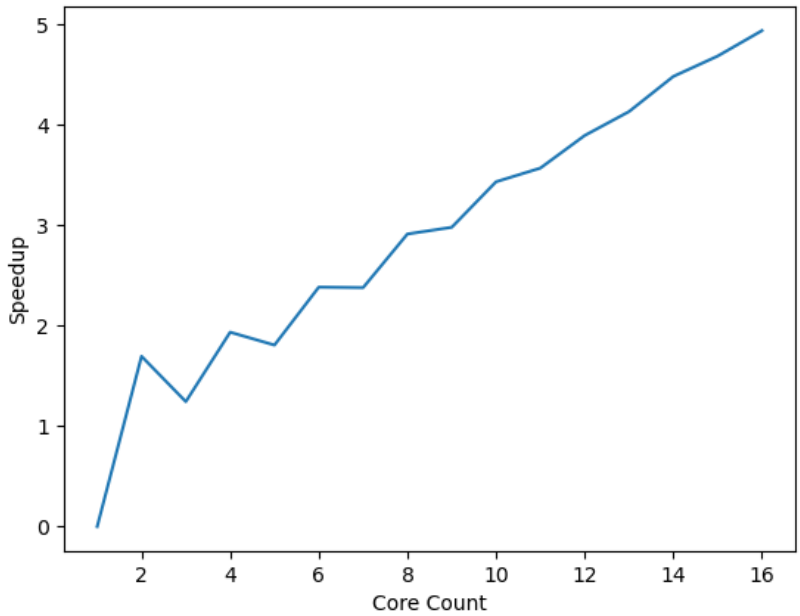

Figure 1, Speedup as a function of the number of cores.

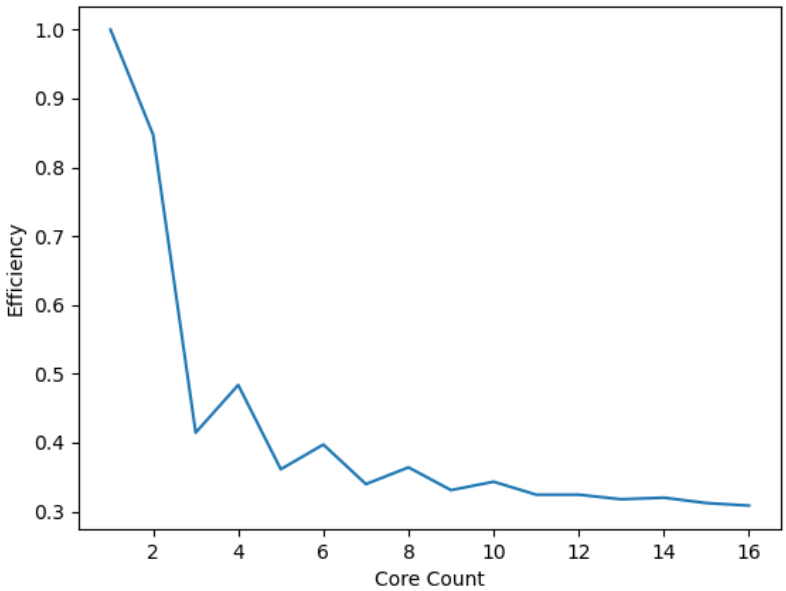

Figure 2, Efficiency as a function of the number of cores.

Both the speedup and efficiency graphs show that the perfect linear speedup is not achieved and with a total of 16 cores the effective speedup is just below 5x the serial implementation.

By looking at the smallest execution time we can calculate the implied serial fraction of the program using the following formula:

$$s = \frac{\frac{1}{S_{p^*}} - \frac{1}{p^*}}{1 - \frac{1}{p^*}}$$

Equation 1

Where p^* indicate the number of processes / cores where the program execution time is the best. For the test results in Table 1, the best execution time at 0.9099 seconds is when 16 cores are used with a speedup of 4.9368. We can now calculate the implied serial fraction and we get the following:

$$s = \frac{\frac{1}{4.9368} - \frac{1}{16}}{1 - \frac{1}{16}} = 0.1494$$

We can now see that the implied serial fraction of the program is ~15%.

L5 – Multiprocessing Chunked

Table 2 shows the sweep results for the multi processed Mandelbrot set program when the chunk size is dynamic. The sweep was performed with a Mandebrot set resolution of 8192x8192 and with 16 workers / cores as this was found to be the optimal number in L4.

Number of chunks	Parallel Time [s]	Serial time [s]	Speedup	LIF
4	2.9458	4.5060	1.5296	9.4560
8	2.4239	4.5060	1.8590	7.6069
16	0.9111	4.5060	4.9459	2.2335
32	0.5531	4.5060	8.1472	0.9639
64	0.6007	4.5060	7.5015	1.1329
128	0.6079	4.5060	7.4122	1.1586
256	0.7511	4.5060	5.9992	1.6670
512	0.7992	4.5060	5.6384	1.8377
1024	0.7623	4.5060	5.9111	1.7068
2048	0.7608	4.5060	5.9225	1.7016
4096	0.7876	4.5060	5.7215	1.7965
8192	0.8720	4.5060	5.1672	2.0965

Table 2, Parallel execution time, serial time, speedup, LIF and implied serial portion measured for 4 to 8192 number of chunks.

By looking at Table 2, we can see that the best result is when splitting the set into 32 row chunks. We can now back-solve the implied serial fraction using Equation 1. We get the following:

$$s = \frac{\frac{1}{8.1472} - \frac{1}{16}}{1 - \frac{1}{16}} = 0.0643$$

We can now see that the implied serial fraction of the program is ~6.4%, this means that the chunked multi processed program improves the load balancing by 8.6 percent points. Likewise, the Load imbalance factor has also decreased by a lot and is now just 96%.

The implementation of the Mandelbrot set with Numba and parallel=True with a resolution of 8192x8192, takes 1.1681 seconds, which means that the load balanced multi-processed solution has a speedup slightly above 2 when comparing against Numba.

L6 – Dask Local

Sweeping was performed on a Mandelbrot set with a resolution of 8192x8192 from 4 chunks to 8192 chunks with a doubling of the chunk size each loop. The timing was recorded using the median of 3 runs using 8 workers, as this is the number of physical cores my AMD Ryzen 7 4800HS processor has. It should be noted that this is slightly different from the multiprocessing implementation which also used the logical cores. The reason for this is that Dask did not seem to utilize the logical cores and the parallel time for 128 chunks with 16 cores was 2.97 seconds which is nearly the same as with just the 8 physical cores as shown below. Table 2 lists the metrics that were recorded during the sweep.

Number of chunks	Parallel Time [s]	Serial time [s]	Speedup	LIF
4	3.8483	4.7458	1.2464	5.4184
8	3.0660	4.7458	1.5645	4.1136
16	3.1897	4.7458	1.5038	4.3199
32	2.7975	4.7458	1.7146	3.6659
64	2.8578	4.7458	1.6784	3.7664
128	2.7796	4.7458	1.7256	3.6360
256	3.1799	4.7458	1.5084	4.3036
512	4.4055	4.7458	1.0772	6.3477
1024	7.2160	4.7458	1.0888	11.0353
2048	13.3141	4.7458	0.6647	21.2060
4096	28.5489	4.7458	0.3603	46.6154
8192	60.6467	4.7458	0.0791	100.1497

Table 3, Parallel execution time, serial time, speedup and LIF measured for 4 to 8192 number of chunks.

Figure 3 shows time as a function of the number of chunks.

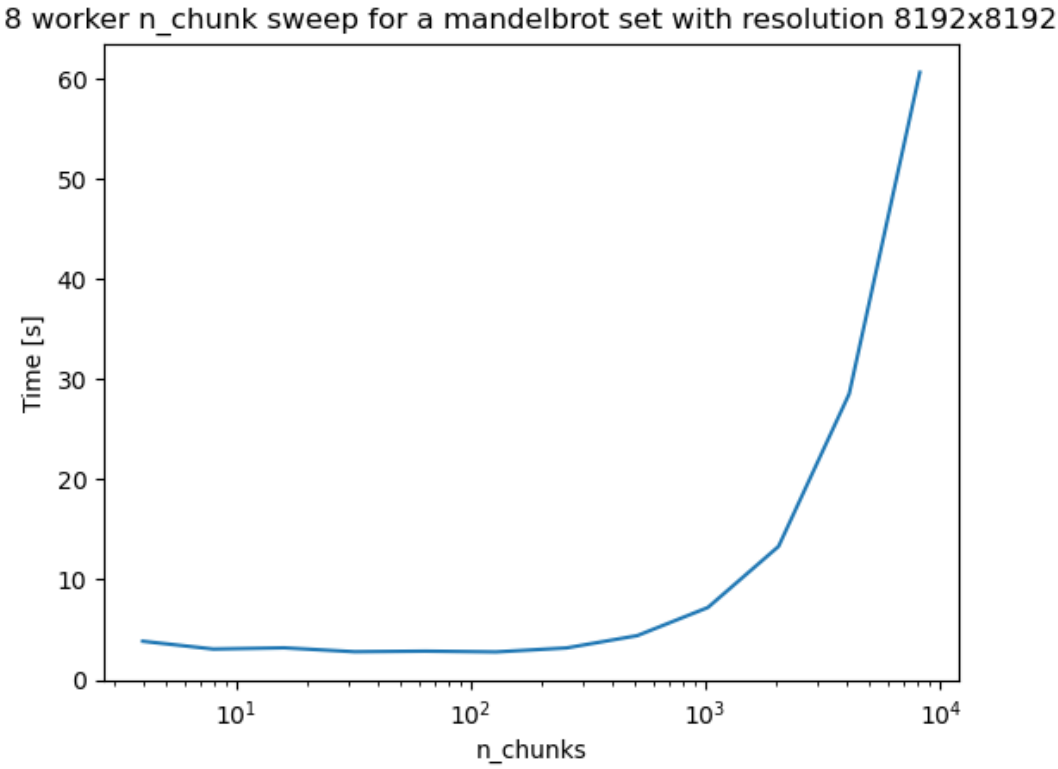

Figure 3, Time as a function of the number of chunks.

By observing Table 3 and looking at Figure 3 we can conclude that minimum execution time and Load Imbalance Factor (LIF) are 2.78s and 364% respectively. Therefore, the optimal number of chunks is 128 with each chunk being 8192x64 pixels in size, when the resolution of the Mandelbrot set is 8192x8192 and 8 workers are used.

It is also quite clearly seen that at and above 2048 chunks the overhead becomes too great and the execution time becomes slower than the serial implementation.

How does Dask local compare to multiprocessing at the same worker count?

The multi-processed implementation from L4 takes 1.70 seconds when 8 workers are used. This is a little faster than the Dask implementation of 2.78 seconds with the same number of workers. However, multi-processed implementation can only be used locally on single machine while Dask enables distributed computing across multiple machines on a network.

What does the overhead difference tell you about when to choose each tool?

The overhead is clearly larger when using Dask, which is why there is a little more than 1 second difference between the implementations. This makes sense as Dask must inherently have more overhead to accommodate distributed computing across multiple networked machines.

L7 – Dask Cluster